LifeDrop

BLOOD DONATION NETWORK

Antigravity Build Guide

Project details, tech stack, features, theme and 18 step-by-step prompts.

Donor <-> Recipient | No hospital, no blood bank

Tech used in this project

MOBILE APP (APK)	BACKEND	DATABASE	WEB (OPTIONAL)
React Native + Expo	Spring Boot 3 + JWT	PostgreSQL	React + Vite + Tailwind

Theme colors (from your screenshot)



Is PDF me kya hai

Ye guide aapko **Antigravity** me LifeDrop project bana kar dene ke liye hai. Pehle project ki puri details (tech, features, theme, structure), phir **ek-ek karke 18 copy-paste prompts**. Prompts English me diye hain kyunki AI tools English me behtar kaam karte hain; samjhane wali baatein Hinglish me hain.

Section	Kya milega
1. Project Overview	Project kya hai, kaun use karega, kya nahi hoga
2. Tech Stack	Har layer ki technology
3. Features	Saare features role ke hisaab se
4. Theme / Design System	Screenshot se nikale gaye colors, fonts, button styles + kya badalna hai
5. Folder Structure	Backend, Web, Mobile ka structure
6. Database and API	Tables aur saare endpoints
7. Antigravity kaise use karein	Tips aur sahi workflow
8. Step-by-step Prompts	Prompt 0 se 17, copy-paste ready
9. Common Issues	Aksar aane wali problems ke solution
10. Final Checklist	Release se pehle check karne ki list

Prompt Map

#	Prompt	Kahan
0	Master Context (sabse pehle ye bhejo)	Setup
1	Backend Project Setup	Backend
2	Entities, Enums and Repositories	Backend
3	Authentication and JWT Security	Backend
4	Profile, Location and Donor Availability	Backend
5	Blood Compatibility and Nearby Donor Search	Backend
6	Blood Request Flow (Create, Accept, Reject, Complete)	Backend
7	Notifications (In-app + Push)	Backend
8	Safety, Seed Data, Swagger and Tests	Backend
9	Web: Setup, Theme and Landing Page	Web (optional)
10	Web: Auth, Dashboards, Map	Web (optional)
11	Mobile: Expo Setup, Theme, Navigation and Auth	Mobile
12	Mobile: Donor Screens	Mobile
13	Mobile: Recipient Screens and Nearby Map	Mobile
14	Mobile: Location Updates, Push Notifications, Notification Screen	Mobile
15	Emergency SOS Feature (Backend + Web + Mobile)	Full-stack
16	Full QA, Bug Fixing and UI Polish	Full-stack
17	APK Build and Deployment	Release

1. Project Overview

LifeDrop ek Blood Donation Network hai jo sirf **do type ke users** ko connect karta hai: **Donor** (jo khoon dena chahta hai) aur **Recipient** (jise khoon ki zaroorat hai). Beech me koi hospital ya blood bank nahi hai. Recipient request daalta hai, system paas ke compatible donors dhoondhta hai, donor accept karta hai, aur accept ke baad hi donor ka phone number recipient ko dikhta hai.

Poora flow

Step	Kya hota hai
1	Donor register karta hai: blood group, city, location, availability.
2	Recipient register karta hai aur login karta hai.
3	Recipient **blood request** banata hai (ya ek tap me **Emergency SOS**).
4	Backend compatible blood group + nearby donors (Haversine, 10 km ya SOS me 25 km) nikalta hai.
5	Un donors ko push notification + in-app notification jaata hai.
6	Jo donor **pehle accept** kare, wahi match hota hai. Baaki requests expire ho jaati hain.
7	Accept ke baad dono ko ek-doosre ka contact dikhta hai (Call / WhatsApp).
8	Recipient **Mark Completed** karta hai. Donor ki last donated date update hoti hai aur 90 din tak wo request nahi paata.

Is project me kya NAHI hoga (Antigravity ko bhi ye bata dena)

- Hospital login ya hospital listing
- Blood bank login, stock ya inventory
- Trauma centers, certifications (NABH/CDSCO) jaise claims
- Admin panel (zaroorat padi to baad me add kar sakte ho)

Disclaimer jo app me dikhana hai

- "LifeDrop connects donors and recipients. In a medical emergency, contact a hospital or call 112."

2. Tech Stack

Layer	Technology	Kaam
Mobile App (APK)	React Native + Expo (JavaScript)	Android app, Expo Go se live test, EAS se APK build
Navigation	React Navigation (stack + bottom tabs)	Role ke hisaab se alag tabs
Backend	Java 17 + Spring Boot 3.x + Maven	REST API, business logic
Security	Spring Security + JWT (jjwt) + BCrypt	Login, role-based access, password hashing
Database	PostgreSQL	Users, requests, responses, notifications
ORM	Spring Data JPA (Hibernate)	Java classes se tables
Nearby search	Haversine formula in native SQL	Radius ke andar ke donors
Location	expo-location	GPS latitude / longitude
Map (mobile)	react-native-maps	Donor pins aur distance
Push	expo-notifications + Expo Push API	Donor ko turant alert
Token storage	expo-secure-store	JWT safe storage
HTTP client	Axios	Mobile aur web se API calls
API docs / test	springdoc Swagger + Postman	Endpoints test karna
Web (optional)	React 18 + Vite + Tailwind CSS + lucide-react	Landing page (aapke screenshot jaisa) + dashboards
Web map	Leaflet + OpenStreetMap	Free map, API key nahi chahiye
Deploy	Render / Railway + Neon / Supabase (free tier)	Backend + PostgreSQL hosting
Build	EAS Build (Expo)	APK (testing) aur AAB (Play Store)

Dhyan do

- Aapka screenshot **web** (localhost:5173, Vite) ka hai, isliye web ka landing page isi look me banega. APK ke liye asli app **React Native** me banega.
- Sirf APK chahiye to Prompt 9 aur 10 (Web) skip kar sakte ho.

3. Features

A. Login and Security (dono users)

- Register / Login / Logout
- Role selection: Donor ya Recipient
- JWT token + BCrypt hashing
- Role-based API access (donor ke routes recipient nahi khol sakta)
- Token secure storage (SecureStore)

B. Donor Features

- Profile: naam, age, blood group, city, phone
- Availability ON / OFF toggle
- 90-din gap rule (donation ke baad auto unavailable)
- Incoming requests list
- Accept / Reject (first donor wins)
- Recipient ko call / WhatsApp (accept ke baad)
- Donation history

C. Recipient Features

- Blood request: patient, group, units, urgency, location, message
- Nearby donors map + list (distance ke saath)
- Request status timeline: Pending, Accepted, Completed
- Accepted donor ka contact + Call button
- Cancel request / Mark completed
- Apni requests ki history

D. Nearby Location

- GPS se latitude/longitude
- Radius search 5 / 10 / 25 km
- Blood group compatibility matching
- Distance display: "3.2 km away"
- Map pins (react-native-maps / Leaflet)

E. Emergency SOS

- Ek tap me URGENT request
- Wider radius (25 km, zarurat par 50 km)
- High-priority push notification
- Live status screen: kitne donors notified
- Quick call: 112 / 108

F. Notifications

- Push: naya request, accept, cancel, complete
- In-app notification list + unread count
- Tap par sahi screen khulti hai

G. Safety and Privacy

- Phone number accept ke baad hi dikhta hai
- Fake request report button (3 reports = auto cancel)
- Din me max 3 requests (rate limit)
- 24 ghante me purani request auto-expire
- Medical disclaimer + Privacy Policy
- Account delete option (Play Store requirement)

Blood Group Compatibility (matching logic)

Recipient	Donor kaun ho sakta hai	Recipient	Donor kaun ho sakta hai
A+	A+, A-, O+, O-	AB+	Sabhi 8 groups
A-	A-, O-	AB-	AB-, A-, B-, O-
B+	B+, B-, O+, O-	O+	O+, O-
B-	B-, O-	O-	Sirf O-

4. Theme / Design System

Ye colors aur style aapke screenshot (LifeDrop landing page) se nikale gaye hain: **crimson red + soft pink + white + dark navy**. Master Context prompt me ye sab already likha hua hai, taaki Antigravity har screen me wahi theme use kare.

	Naam	Hex	Kahan use hoga
	Primary Dark	#991B1B	Register button, headline red, logo background
	Primary	#B91C1C	Active links, icons, active tab
	Accent / SOS	#DC2626	SOS button, urgent badge, highlights
	SOS Gradient End	#BE123C	SOS button gradient ka end
	Top Bar Red	#C62828	Top alert strip, button gradient end
	Page Background	#FFF5F5	Hero aur page background (light pink)
	Soft Pink	#FEE2E2	Active nav pill, chips, light fills
	Border Pink	#FECACA	Card aur secondary button border
	Ink (Navy)	#0F172A	Headings, main text
	Muted Slate	#475569	Subtitle, body text
	Light Muted	#94A3B8	Placeholder, tagline, small text
	Success	#16A34A	Accepted badge, success toast
	Warning	#F59E0B	Pending badge

Typography, shape aur components

Item	Style
Font	**Plus Jakarta Sans** (Google Fonts), weights 400 to 800. Headings 800, tight letter-spacing.
Hero headline	Do line: "Every Drop Counts." navy + "Every Life Saved." red gradient
Radius	Buttons 12px, cards 20px, badges full pill
Shadow	Red soft glow: 0 10px 30px rgba(185,28,28,0.30)
Primary button	Gradient #991B1B to #C62828, white bold text
Secondary button	#FFF1F1 background, red text, 1px #FECACA border, heart icon
SOS button	Gradient #DC2626 to #BE123C, alert-triangle icon, glow shadow
Navbar	White, sticky, 1px bottom border; active link = pink pill with red text
Top bar	Red gradient strip, white small bold text, "LIVE" pill
Logo	Rounded-square dark-red gradient + white blood-drop; "Life" navy + "Drop" red; tagline in slate
Icons	lucide-react (web), @expo/vector-icons (mobile)

Screenshot me kya BADALNA hai (zaroori)

- Badge **"NABH and CDSCO Certified Clinical Transfusion Network"** hatao. Aapke paas certification nahi hai, ye jhootha claim hoga. Naya: **"Community-driven Blood Donor Network"**.
- Subtitle me **"trauma centers and certified blood banks"** hatao (project me hospital / blood bank nahi hai). Naya: "connecting voluntary blood donors with people who need blood".
- Top bar: **"National Emergency Transfusion Helpline 108 / 104"** ki jagah **"Medical emergency? Call 112 / 108"**.
- Navbar se **Admin** link hatao (sirf 2 users hain).
- Rakhna hai: logo, colors, hero layout, 3 CTA buttons (Find Donors Near Me, Register as a Donor, EMERGENCY SOS), Sign In / Register.

5. Folder Structure

Backend (Spring Boot)

```
lifedrop/
  backend/
    pom.xml
    src/main/java/com/lifedrop/
      BackendApplication.java
      config/ SecurityConfig, CorsConfig, SchedulingConfig
      controller/ AuthController, UserController, DonorController,
                  RecipientController, NotificationController, SosController
      dto/ RegisterRequest, LoginRequest, AuthResponse, ApiResponse,
                  BloodRequestDto, DonorResponseDto, NotificationDto
      entity/ User, BloodRequest, RequestResponse, Notification, Report
      enums/ Role, BloodGroup, RequestStatus, ResponseStatus, Urgency
      exception/ GlobalExceptionHandler, BadRequestException, ...
      repository/ UserRepository, BloodRequestRepository,
                  RequestResponseRepository, NotificationRepository
      security/ JwtUtil, JwtAuthFilter, CustomUserDetailsService
      service/ AuthService, UserService, MatchingService,
                  BloodRequestService, NotificationService
      util/ BloodCompatibility, DistanceCalculator
    src/main/resources/
      application.properties, application-dev.properties
    src/test/java/... (compatibility, matching, request flow tests)
```

Mobile App (React Native + Expo)

```
lifedrop/mobile/
  App.js app.json eas.json .env
  assets/ (icon, adaptive-icon, splash)
  src/
    api/ axiosInstance, authApi, requestApi, userApi, notificationApi
    context/ AuthContext
    navigation/ AppNavigator, AuthStack, DonorTabs, RecipientTabs
    screens/
      auth/ LoginScreen, RegisterScreen
      donor/ DonorHomeScreen, IncomingRequestsScreen,
                RequestDetailScreen, DonationHistoryScreen
      recipient/ RecipientHomeScreen, CreateRequestScreen, NearbyDonorsScreen,
                MyRequestsScreen, RequestStatusScreen, SosScreen
      common/ SplashScreen, ProfileScreen, NotificationScreen
    components/ CustomButton, InputField, BloodGroupPicker, Card, StatusBadge,
                DonorCard, RequestCard, Loader, EmptyState, ScreenHeader
    hooks/ useLocation, useNotifications
    theme/ colors, typography, spacing
    utils/ constants, validators
```

Web (React + Vite) - optional

```
lifedrop/web/
  index.html vite.config.js tailwind.config.js .env
  src/
    api/ axiosInstance, authApi, requestApi
    context/ AuthContext
    components/ TopAlertBar, Navbar, Footer, Logo, Button, BloodGroupChips,
                DonorCard, RequestCard, MapView, SosModal, ProtectedRoute
    pages/ Home, HowItWorks, Login, Register, FindDonors, BloodRequests,
                DonorDashboard, RecipientDashboard, Profile, Privacy, Terms, NotFound
    hooks/ useGeolocation, useNotifications
    utils/ constants, formatters
```

6. Database and API

Database Tables (PostgreSQL)

Table	Columns
users	id, name, email (unique), password (hash), phone, role (DONOR / RECIPIENT), blood_group, age, gender, city, latitude, longitude, available, last_donated_date, push_token, active, created_at
blood_requests	id, recipient_id (FK), patient_name, blood_group, units_needed, urgency (NORMAL / URGENT), city, latitude, longitude, radius_km, message, required_by, status (PENDING / ACCEPTED / COMPLETED / CANCELLED / EXPIRED), accepted_donor_id (FK), created_at
request_responses	id, request_id (FK), donor_id (FK), status (PENDING / ACCEPTED / REJECTED / EXPIRED), responded_at. Unique (request_id, donor_id)
notifications	id, user_id (FK), title, body, type, related_request_id, read, created_at
reports	id, reporter_id (FK), request_id (FK), reason, created_at

Relations: ek recipient ki kai requests; ek request kai donors ko jaati hai (request_responses); jo donor pehle accept kare wo blood_requests.accepted_donor_id me save hota hai.

API Endpoints

Method	Endpoint	Kaun	Kaam
POST	/api/auth/register	Public	Naya user register
POST	/api/auth/login	Public	Login, JWT token
GET	/api/auth/me	Login	Current user
GET / PUT	/api/users/me	Login	Profile dekhna / badalna
PUT	/api/users/location	Login	Latitude / longitude update
PUT	/api/users/push-token	Login	Push token save
DELETE	/api/users/me	Login	Account delete (anonymize)
PUT	/api/donor/availability	Donor	Available ON / OFF
GET	/api/donor/eligibility	Donor	90-din rule check
GET	/api/donor/requests	Donor	Incoming requests
GET	/api/donor/requests/{id}	Donor	Request details
POST	/api/donor/requests/{id}/accept	Donor	Accept (first wins)
POST	/api/donor/requests/{id}/reject	Donor	Reject
GET	/api/donor/history	Donor	Donation history
POST	/api/recipient/requests	Recipient	Blood request banana
GET	/api/recipient/requests	Recipient	Meri requests
GET	/api/recipient/requests/{id}	Recipient	Details (accept ke baad donor phone)
PUT	/api/recipient/requests/{id}/cancel	Recipient	Cancel
POST	/api/recipient/requests/{id}/complete	Recipient	Mark completed
GET	/api/recipient/donors/nearby	Recipient	Nearby donors (map + list)
POST	/api/recipient/sos	Recipient	Emergency SOS
POST	/api/requests/{id}/report	Login	Fake request report
GET	/api/notifications	Login	Notifications list
GET	/api/notifications/unread-count	Login	Unread count
PUT	/api/notifications/{id}/read	Login	Read mark
PUT	/api/notifications/read-all	Login	Sab read

7. Antigravity kaise use karein

Step	Kya karna hai
1	Ek **khali folder** (lifedrop) Antigravity me workspace ki tarah kholo.
2	Sabse pehle **Prompt 0 (Master Context)** bhejo. Ye project ki yaaddasht hai.
3	Phir **sirf ek prompt ek baar** me bhejo, number order me (1, 2, 3...). Ek saath do prompt mat do.
4	Complex prompts (3, 6, 10, 13) me **Planning mode** use karo aur Antigravity ka plan padh kar approve karo.
5	Har prompt ke baad **run karo aur test karo**. Tabhi agla prompt bhejo.
6	Har step ke baad **git commit** karo. Kuch bigde to purane step par wapas aa sakte ho.
7	Error aaye to poora error text paste karo aur likho: *"Fix this error. Do not change unrelated files."*
8	Design alag dikhe to likho: *"Match the DESIGN SYSTEM from the Master Context exactly. List what differs and fix it."*
9	Naya chat khole to Master Context dobara paste karo aur likho kaun sa prompt number ho chuka hai.

Copy-paste ka tip

- PDF se copy karne par line breaks beech me aa sakte hain. Isse prompt ke matlab par koi fark nahi padta.
- Saaf copy ke liye saath me diya gaya **LifeDrop_Prompts.md** file use karo.

Prompt order ek nazar me

Phase	Prompts	Result
Setup	0	Antigravity ko project ka context mil gaya
Backend	1 to 8	Spring Boot API poori tarah ready (Swagger se test)
Web (optional)	9 to 10	Screenshot jaisa landing page + dashboards
Mobile	11 to 14	React Native app: login, donor, recipient, map, push
Full-stack	15	Emergency SOS
Release	16 to 17	QA, deployment, APK

8. Step-by-step Prompts

Har prompt ke andar jo **grey-pink box** hai, wahi Antigravity me paste karna hai. Box ke neeche "Check karo" list hai; wo pura hone ke baad hi agle prompt par jao.

PROMPT 0 | Master Context (sabse pehle ye bhejo)

Area: Setup **Kab bhejna hai:** Naya chat / workspace kholne ke baad sabse pehla message. Isme code mat likhwana, sirf samjhane do.

ROLE
You are a senior full-stack engineer and UI designer. We will build a project called "LifeDrop" step by step. Read this whole context carefully and remember it for every next prompt. Do NOT write any code in this message. Reply only with a short summary of your understanding and the folder plan.

PRODUCT
LifeDrop is a Blood Donation Network that connects ONLY two kinds of users:
1. DONOR - a person who is willing to donate blood.
2. RECIPIENT - a person (or family member) who needs blood.
There are NO hospitals, NO blood banks, NO trauma centers and NO admin-managed blood inventory in this product. Never add them anywhere (UI text, database, API, images).
Core idea: a recipient creates a blood request, the system finds compatible donors nearby, donors accept or reject, and only after a donor accepts, the donor's phone number is shown to the recipient.

TECH STACK
- Backend: Java 17, Spring Boot 3.x, Maven, Spring Security + JWT (jjwt) + BCrypt, Spring Data JPA (Hibernate), Lombok, springdoc-openapi (Swagger).
- Database: PostgreSQL.
- Mobile app (APK): React Native with Expo (JavaScript), React Navigation, Axios, expo-location, expo-secure-store, expo-notifications, react-native-maps.
- Web (optional): React 18 + Vite + React Router + Axios + Tailwind CSS + lucide-react icons + Leaflet (OpenStreetMap).
- Nearby search: Haversine formula inside a native SQL query.

FOLDER STRUCTURE
lifedrop/
  backend/ (Spring Boot)
  web/ (React + Vite)
  mobile/ (React Native + Expo)
  README.md

CORE FEATURES
- Auth: register, login, logout, role selection (DONOR / RECIPIENT), JWT, BCrypt password hashing.
- Donor: profile, availability ON/OFF, 90-day gap after donation, incoming requests, accept/reject, donation history.
- Recipient: create blood request, see nearby donors (map + list with distance), track request status, see donor contact after accept, mark request completed.
- Nearby location: GPS latitude/longitude, radius search, blood group compatibility matching.
- Emergency SOS: one-tap URGENT request with a wider search radius.
- Notifications: in-app notifications + push notifications.
- Safety: phone number hidden until accept, report fake request, daily request limit, medical disclaimer.

DESIGN SYSTEM (use in EVERY UI screen)
Style: clean, modern, friendly medical look. Light pink page background, white cards, crimson red accents, dark navy text.
Colors: primary-dark #991B1B, primary #B91C1C, accent/SOS #DC2626, SOS-gradient-end #BE123C, page-bg #FFF5F5, soft-pink #FEE2E2, border-pink #FECACA, ink #0F172A, muted #475569, light-muted #94A3B8, white #FFFFFF, success #16A34A, warning #F59E0B.
Font: Plus Jakarta Sans (weights 400 to 800). Headings use weight 800 with tight letter spacing.
Shape: buttons radius 12px, cards radius 20px, badges fully rounded (pill). Red soft glow shadow on primary and SOS buttons: 0 10px 30px rgba(185,28,28,0.30).
Buttons: Primary = gradient #991B1B to #C62828 with white text. Secondary = #FFF1F1 background, red text, 1px #FECACA border, heart icon. SOS = gradient #DC2626 to #BE123C with alert icon and glow.

Logo: rounded-square dark red gradient with a white blood-drop icon, text "Life" in navy + "Drop" in red, and a small slate tagline "BLOOD DONATION NETWORK".

WORKING RULES
1. Do only what the current prompt asks. Do not touch unrelated files.
2. Before coding, show a short plan. After coding, list created/changed files, the command to run, and how to test.
3. Layered architecture (Controller -> Service -> Repository), DTOs for all requests/responses, validation on inputs. Never return the password hash. Never return a phone number before a donor accepts.
4. No hard-coded secrets. Use environment variables.
5. Code must compile and run. Fix your own errors before finishing.
6. UI text must be simple, friendly English. No fake claims (no certifications, no fake statistics, no hospital or blood-bank wording).
7. Mobile first: every screen must look good on a 360px wide phone.

Check karo

- Antigravity ne bina code likhe summary di ho.
- Summary me 'no hospital / no blood bank' wali baat likhi ho.
- Agar Antigravity me Rules / Workspace instructions ka option hai to ye Master Context wahan bhi save kar do.

PROMPT 1 | Backend Project Setup

Area: Backend **Kab bhejna hai:** Master Context ke baad. Pehle PostgreSQL install kar lo aur `lifedrop` naam ka database bana lo.

TASK
Create the Spring Boot backend in lifedrop/backend.

REQUIREMENTS
- Maven project, Java 17, Spring Boot 3.x, group com.lifedrop, artifact backend.
- Dependencies: spring-boot-starter-web, data-jpa, security, validation, postgresql driver, lombok, springdoc-openapi-starter-webmvc-ui, jjwt-api + jjwt-impl + jjwt-jackson (0.12.x).
- Create these packages under com.lifedrop: config, controller, dto, entity, enums, exception, repository, security, service, util.
- application.properties must read values from environment variables with local defaults: DB_URL (jdbc:postgresql://localhost:5432/lifedrop), DB_USER (postgres), DB_PASS, JWT_SECRET, JWT_EXPIRATION (86400000), server.port=8080, spring.jpa.hibernate.ddl-auto=update, spring.jpa.open-in-view=false.
- Add CorsConfig allowing all origins for development.
- Add HealthController: GET /api/health returns {"status":"UP"}.
- Add a proper .gitignore (target/, .env, application-local.properties, firebase json files).

ALSO EXPLAIN
- How to create the PostgreSQL database named lifedrop.
- The exact command to run the app.

ACCEPTANCE
App starts on port 8080 and GET http://localhost:8080/api/health returns status UP. Swagger UI opens at /swagger-ui.html.

Check karo

- Terminal me app bina error ke start ho.
- Browser me /api/health aur /swagger-ui.html khulte hain.
- Git commit kar do: 'step 1 backend setup'.

PROMPT 2 | Entities, Enums and Repositories

Area: Backend **Kab bhejna hai:** Setup chalne ke baad. Ye database ki tables banata hai.

TASK
Create enums, JPA entities and Spring Data repositories in lifedrop/backend.

ENUMS

- Role: DONOR, RECIPIENT
- BloodGroup: A_POSITIVE, A_NEGATIVE, B_POSITIVE, B_NEGATIVE, AB_POSITIVE, AB_NEGATIVE, O_POSITIVE, O_NEGATIVE. Each has a display label such as "A+" and a static fromLabel() helper.
- RequestStatus: PENDING, ACCEPTED, COMPLETED, CANCELLED, EXPIRED
- ResponseStatus: PENDING, ACCEPTED, REJECTED, EXPIRED
- Urgency: NORMAL, URGENT

ENTITIES (all with createdAt)
1. User (table users): id, name, email (unique), password, phone, role, bloodGroup, age, gender, city, latitude, longitude, available (default true), lastDonatedDate, pushToken, active (default true), createdAt.
2. BloodRequest (table blood_requests): id, recipient (ManyToOne User), patientName, bloodGroup, unitsNeeded, urgency, city, latitude, longitude, radiusKm (default 10), message, requiredBy (LocalDateTime), status, acceptedDonor (ManyToOne User, nullable), createdAt.
3. RequestResponse (table request_responses): id, request (ManyToOne), donor (ManyToOne), status, respondedAt. Unique constraint on (request_id, donor_id).
4. Notification (table notifications): id, user (ManyToOne), title, body, type, relatedRequestId, read (boolean), createdAt.

REPOSITORIES
UserRepository (findByEmail, existsByEmail), BloodRequestRepository, RequestResponseRepository, NotificationRepository with sensible finder methods (for example by recipient, by donor and status, by user ordered by createdAt desc).

RULES
Use Lombok, LAZY fetching, and add indexes on users(role, blood_group, available) and blood_requests(status).

ACCEPTANCE
App starts and Hibernate creates all 4 tables in PostgreSQL without errors.

Check karo

- pgAdmin / DBeaver me users, blood_requests, request_responses, notifications tables dikhni chahiye.
- Git commit: 'step 2 entities'.

PROMPT 3 | Authentication and JWT Security

Area: Backend **Kab bhejna hai:** Tables ban jaane ke baad. Login system yahin banta hai.

TASK
Build register/login with JWT and role-based security.

DTOs
- RegisterRequest: name, email, password (min 8), phone (10 digits), role, bloodGroup (REQUIRED when role = DONOR), age (donor must be 18 to 65), gender, city, latitude, longitude.
- LoginRequest: email, password.
- AuthResponse: token, and a UserSummary (id, name, email, role, bloodGroup, city, available).
- ApiResponse<T> wrapper: success, message, data.

ENDPOINTS
- POST /api/auth/register
- POST /api/auth/login
- GET /api/auth/me (needs token)

SECURITY
- BCrypt password encoder.
- JwtUtil (generate, validate, read email and role) using JWT_SECRET and JWT_EXPIRATION from properties.
- JwtAuthFilter reading "Authorization: Bearer <token>".
- CustomUserDetailsService.
- SecurityConfig: stateless sessions, CSRF disabled, public routes = /api/auth/**, /api/health, swagger URLs. /api/donor/** only for DONOR. /api/recipient/** only for RECIPIENT. Everything else needs login.
- Return clean JSON 401/403 errors (no HTML pages).
- GlobalExceptionHandler with ResourceNotFoundException, BadRequestException, validation errors (field-wise messages) and a generic fallback.

ACCEPTANCE
Register donor, register recipient, login, call /api/auth/me with token. Wrong password returns 401. Donor calling a /api/recipient/** route returns 403. Show me sample Postman/curl requests.

Check karo

- Postman me register + login + /me test karo.
- Password DB me hashed (BCrypt) dikhna chahiye, plain nahi.
- Git commit: 'step 3 auth'.

PROMPT 4 | Profile, Location and Donor Availability

Area: Backend **Kab bhejna hai:** Auth chalne ke baad.

TASK
Add profile, location and donor availability APIs.

ENDPOINTS
- GET /api/users/me - full profile of logged-in user.
- PUT /api/users/me - update name, phone, city, age, gender, bloodGroup (donor), never email or role.
- PUT /api/users/location - body {latitude, longitude}; validate ranges (-90..90, -180..180).
- PUT /api/users/push-token - body {pushToken}.
- PUT /api/donor/availability - body {available: true/false}.
- GET /api/donor/eligibility - returns {eligible, lastDonatedDate, nextEligibleDate, daysRemaining}.

BUSINESS RULE
Put donation.gap-days=90 in application.properties. A donor is eligible only if lastDonatedDate is null or at least 90 days ago. If a donor who is not eligible tries to turn availability ON, return a 400 error with a clear message and the next eligible date.

ACCEPTANCE
Location and availability update correctly. Eligibility returns correct days remaining. Add 3 unit tests for the eligibility logic.

Check karo

- Donor ka last donated date manually DB me set karke eligibility test karo.
- Git commit: 'step 4 profile location'.

PROMPT 5 | Blood Compatibility and Nearby Donor Search

Area: Backend **Kab bhejna hai:** Ye project ka dil hai (matching logic). Dhyan se test karna.

TASK
Build blood compatibility and nearby donor search.

BloodCompatibility (util class)
Given the RECIPIENT blood group, return the list of DONOR groups that can donate:
A+ <- A+, A-, O+, O-
A- <- A-, O-
B+ <- B+, B-, O+, O-
B- <- B-, O-
AB+ <- all 8 groups
AB- <- AB-, A-, B-, O-
O+ <- O+, O-
O- <- O- only

DistanceCalculator
Haversine distance in km between two lat/lng points.

UserRepository
Add a native query findNearbyDonors(groups, lat, lng, radiusKm, eligibleBefore) that returns DONORS where: role = 'DONOR', active = true, available = true, blood_group IN groups, latitude/longitude not null, (last_donated_date IS NULL OR

last_donated_date <= eligibleBefore), and Haversine distance < radiusKm. Order by distance ascending. Also return the distance value (use a projection interface).

MatchingService
findCompatibleDonors(bloodGroup, lat, lng, radiusKm) using the above and donation.gap-days.

ENDPOINT
GET /api/recipient/donors/nearby?bloodGroup=&lat=&lng=&radiusKm=10
Response items (DonorResponseDto): id, firstName only, bloodGroup, distanceKm (1 decimal), city, lastDonatedDate. NEVER return phone, email or exact coordinates here.

ACCEPTANCE
Unit tests for every recipient blood group in the compatibility table. Tests for the distance calculator. Explain how the SQL works in 5 lines.

Check karo

- 8 recipient groups ka compatibility test pass hona chahiye.
- Response me phone / email nahi aana chahiye.
- Git commit: 'step 5 matching'.

PROMPT 6 | Blood Request Flow (Create, Accept, Reject, Complete)

Area: Backend **Kab bhejna hai:** Matching ke baad. Ye sabse bada backend step hai; Antigravity ko Planning mode me chalao.

TASK
Build the complete blood request flow.

RECIPIENT ENDPOINTS
- POST /api/recipient/requests - create request (patientName, bloodGroup, unitsNeeded, urgency, city, latitude, longitude, radiusKm, message, requiredBy). Save with status PENDING, then find compatible nearby donors with MatchingService (use radius 10 km for NORMAL and 25 km for URGENT unless radiusKm is given). Create one RequestResponse (PENDING) per matched donor. Call NotificationService.notifyDonorsOfNewRequest() (create an interface + a simple logging implementation now; real push comes later). Return the request plus matchedDonorCount.
- GET /api/recipient/requests - my requests (newest first).
- GET /api/recipient/requests/{id} - details. If status is ACCEPTED or COMPLETED include the accepted donor's name and phone. Otherwise never include any donor phone.
- PUT /api/recipient/requests/{id}/cancel
- POST /api/recipient/requests/{id}/complete - only when ACCEPTED; set status COMPLETED and set the donor's lastDonatedDate = today.

DONOR ENDPOINTS
- GET /api/donor/requests - incoming requests where my response is PENDING.
- GET /api/donor/requests/{id} - details (recipient first name, blood group, city, urgency, message, distance). Recipient phone only if I accepted.
- POST /api/donor/requests/{id}/accept - @Transactional. Only first donor wins: if the request is no longer PENDING return 409. Set acceptedDonor, status ACCEPTED, my response ACCEPTED, all other responses EXPIRED. Return recipient contact.
- POST /api/donor/requests/{id}/reject
- GET /api/donor/history - requests I accepted/completed.

RULES
- A donor can respond only to requests that were matched to him.
- A recipient can manage only his own requests.
- A scheduled job (@Scheduled every 15 minutes) marks PENDING requests older than 24 hours as EXPIRED.
- Enable scheduling with @EnableScheduling.

ACCEPTANCE
Write an integration test for: create request -> donor accepts -> second donor accept fails with 409 -> recipient completes -> donor lastDonatedDate updated. Provide a Postman collection (JSON) for all endpoints.

Check karo

- Phone number sirf accept ke baad dikhna chahiye (dono taraf).
- Do donors ek saath accept karein to sirf pehla jeete.
- Git commit: 'step 6 request flow'.

PROMPT 7 | Notifications (In-app + Push)

Area: Backend **Kab bhejna hai:** Request flow chalne ke baad.

TASK
Implement real notifications in the backend.

NotificationService (replace the logging stub)
- Always save an in-app Notification row for the target user.
- Also send a push notification when the user has a pushToken. Use the Expo Push API (POST https://exp.host/--/api/v2/push/send) with Spring RestClient. Include title, body, sound "default", and data {requestId, type}. For URGENT requests set priority "high" and channelId "urgent".
- If sending fails, log the error and continue. Never fail the main request because of a push error.

WHEN TO NOTIFY
- New request -> every matched donor ("A+ blood needed near you").
- Donor accepts -> the recipient ("A donor accepted your request").
- Request cancelled -> donors who were still PENDING.
- Request completed -> the donor ("Thank you for saving a life").

ENDPOINTS
- GET /api/notifications (paged, newest first)
- GET /api/notifications/unread-count
- PUT /api/notifications/{id}/read
- PUT /api/notifications/read-all

ACCEPTANCE
Show how to test using a real Expo push token later. For now log the outgoing payload and write unit tests with a mocked RestClient.

Check karo

- Request create karne par donors ke notifications table me rows aani chahiye.
- Git commit: 'step 7 notifications'.

PROMPT 8 | Safety, Seed Data, Swagger and Tests

Area: Backend **Kab bhejna hai:** Backend lagbhag complete. Ab usse mazboot banate hain.

TASK
Harden the backend and add demo data.

SAFETY
- Report fake request: POST /api/requests/{id}/report with {reason}. Create a Report entity (id, reporter, request, reason, createdAt). A user cannot report the same request twice. After 3 reports, mark the request CANCELLED.
- Rate limit: a recipient may create at most 3 requests per 24 hours. Return 429 with a clear message.
- Never log passwords or tokens.

DEMO DATA
Create a dev-profile CommandLineRunner (profile "dev") that inserts 12 donors with mixed blood groups around Bhopal, Madhya Pradesh (center 23.2599, 77.4126, spread within 15 km) and 2 recipients. Password for all: Password@123. Emails like donor1@lifedrop.test and recipient1@lifedrop.test. Insert only if the users table is empty.

DOCS AND TESTS
- Swagger annotations: tag every controller, add JWT "Authorize" button support.
- Add README section for backend: setup, env variables, run, test, demo logins.
- Run all tests and fix failures.

ACCEPTANCE
mvn test passes. With the dev profile, a recipient request for A+ near Bhopal matches seeded donors.

Check karo

- Swagger me Authorize button se token daal kar saari APIs chal rahi ho.
- Git commit: 'step 8 safety and seed'. Ab backend ready hai.

PROMPT 9 | Web: Setup, Theme and Landing Page

Area: Web (optional) **Kab bhejna hai:** Agar sirf APK chahiye to Prompt 9 aur 10 skip karke Prompt 11 par jao. Landing page isi theme ka hai jo aapne screenshot me bheja.

TASK
Create the React web app in lifedrop/web.

SETUP
Vite + React 18, React Router, Axios, Tailwind CSS (latest stable), lucide-react, react-leaflet + leaflet. Load Plus Jakarta Sans from Google Fonts. Put all design tokens from the DESIGN SYSTEM in the Master Context into the Tailwind theme / CSS variables. Create folders: src/api, src/components, src/context, src/pages, src/hooks, src/utils.

LAYOUT COMPONENTS
1. TopAlertBar - red gradient strip (#991B1B to #C62828), small white bold text. A rounded "LIVE" pill at left and text: "Medical emergency? Call 112 (Emergency) / 108 (Ambulance)".
2. Navbar - sticky, white, 1px bottom border. Left: logo (rounded-square dark-red gradient with white drop icon, "LifeDrop" with Life navy and Drop red, tagline BLOOD DONATION NETWORK). Center links: Home, How It Works, Find Donors, Blood Requests. The active link has a soft pink pill background (#FEE2E2) and red text. Right: "EMERGENCY SOS" pill button (red gradient, alert icon, glow shadow), a thin divider, "Sign In" text link, "Register" dark red button. On mobile collapse into a hamburger menu. Do NOT add an Admin link.
3. Footer - logo, short links, privacy/terms links, and the disclaimer: "LifeDrop connects donors and recipients. In a medical emergency, contact a hospital or call 112."

LANDING PAGE (Home)
Hero section on #FFF5F5 background, centered:
- Small pill badge: "Community-driven Blood Donor Network" (do NOT say certified).
- H1 in two lines, very bold, tight tracking: "Every Drop Counts." in navy and "Every Life Saved." in red (second line in the primary-dark to accent gradient).
- Subtitle in slate: "A real-time network connecting voluntary blood donors with people who need blood, within seconds during critical emergencies."
- Three buttons in one row: "Find Donors Near Me" (primary + search icon), "Register as a Donor" (secondary + heart icon), "EMERGENCY SOS" (SOS style + alert icon).
Below the hero add: a 4-card stats strip (use neutral labels like Donors, Requests, Cities, Lives helped and show 0 until API data exists, no fake numbers), "How It Works" in 4 steps (Register, Post request, Nearby donors get alerted, Donor accepts and contacts you), a blood group compatibility chart (recipient vs donors), a "Why LifeDrop" 3-card section (Nearby matching, Privacy first, Instant alerts), and a final call-to-action band in red gradient.

ACCEPTANCE
Run npm run dev. The home page must look like a polished version of: white navbar, pink hero, big bold two-color headline, three CTA buttons. It must be responsive at 360px, 768px and 1280px.

Check karo

- localhost:5173 par page screenshot jaisa dikhna chahiye (par certified/blood bank wording ke bina).
- Mobile width par navbar hamburger ban jaye.
- Git commit: 'step 9 web landing'.

PROMPT 10 | Web: Auth, Dashboards, Map

Area: Web (optional) **Kab bhejna hai:** Landing page ready hone ke baad.

TASK
Connect the web app to the backend and build the pages.

FOUNDATION
- src/api/axiosInstance.js with baseURL from VITE_API_URL and an interceptor that adds the JWT token; on 401 log out and redirect to /login.
- AuthContext (user, token, login, register, logout) saving token in localStorage.
- ProtectedRoute with role check (DONOR / RECIPIENT).

PAGES
1. Login and Register: Register has a role toggle (I am a Donor / I need blood). Donor fields: blood group, age, gender, city. Show a "Use my current location" button (browser Geolocation). Show friendly field errors from the API.
2. Donor Dashboard: availability toggle, eligibility card (days remaining after last donation), incoming requests list with Accept/Reject buttons, history list.
3. Recipient Dashboard: "Request Blood" form (patient name, blood group, units, urgency, city, location picker, required-by, message), "My Requests" with a status timeline (Pending, Accepted, Completed), and when accepted show the donor name and a "Call donor" button.
4. Find Donors page (Recipient, after login): map (Leaflet + OpenStreetMap) with donor pins and a list with distance such as "3.2 km away". Blood group and radius filters. Never show phone numbers here.
5. Blood Requests page: list of open requests for donors; guests see only blood group, city, urgency (no names).
6. Profile page, Notifications dropdown in the navbar with unread count (poll every 30 seconds), Privacy Policy and Terms pages, 404 page.

UI RULES
Every page uses the DESIGN SYSTEM. Use skeleton loaders, empty states and toast messages. All forms validate before submit.

ACCEPTANCE
Full flow works in the browser against the running backend: register donor, register recipient, create request, donor accepts, recipient sees phone number.

Check karo

- Do browsers (ek donor, ek recipient) se poora flow test karo.
- Git commit: 'step 10 web pages'.

PROMPT 11 | Mobile: Expo Setup, Theme, Navigation and Auth

Area: Mobile **Kab bhejna hai:** Yahan se APK ka kaam shuru hota hai. Backend chal raha hona chahiye.

TASK
Create the React Native (Expo) app in lifedrop/mobile using JavaScript.

SETUP
Install: @react-navigation/native, native-stack, bottom-tabs, axios, expo-secure-store, expo-location, expo-notifications, expo-linear-gradient, react-native-maps, @expo/vector-icons, @expo-google-fonts/plus-jakarta-sans, expo-font. Use the folder layout:
src/api, src/context, src/navigation, src/screens/{auth,donor,recipient,common}, src/components, src/hooks, src/utils, src/theme.

THEME (src/theme/colors.js, typography.js, spacing.js)
Copy every token from the DESIGN SYSTEM in the Master Context. Load Plus Jakarta Sans (400, 500, 600, 700, 800). Create a shared shadow helper for the red glow.

API
src/api/axiosInstance.js: baseURL from EXPO_PUBLIC_API_URL (document that on a real phone this must be the PC's LAN IP such as http://192.168.x.x:8080 and on the Android emulator http://10.0.2.2:8080). Add a JWT interceptor using expo-secure-store.

NAVIGATION
AuthContext (token, user, role, login, register, logout, loading). AppNavigator: loading -> Splash; no token -> AuthStack (Login, Register); DONOR -> DonorTabs (Home, Requests, History, Profile); RECIPIENT -> RecipientTabs (Home, Request Blood,

My Requests, Profile). Bottom tab bar is white with red active icon and label.

COMPONENTS
CustomButton (variants: primary gradient, secondary, sos, outline), InputField (label, error text, password eye toggle), BloodGroupPicker (8 chips, selected chip is red), Card, StatusBadge (Pending amber, Accepted green, Completed blue-grey, Cancelled/Expired grey, Urgent red), Loader, EmptyState, ScreenHeader.

SCREENS
SplashScreen (pink background, logo and tagline), LoginScreen, RegisterScreen (role toggle at the top, donor-only fields appear for DONOR, "Use my location" button). Hero text on Login: "Every Drop Counts. Every Life Saved."

ACCEPTANCE
Run with npx expo start. Register and login work against the backend; after login the correct tabs appear for each role; logout works; token survives app restart.

Check karo

- Expo Go me Login/Register chal rahe hon.
- Phone aur PC same Wi-Fi par hon, aur EXPO_PUBLIC_API_URL me PC ka IP ho.
- Git commit: 'step 11 mobile setup'.

PROMPT 12 | Mobile: Donor Screens

Area: Mobile **Kab bhejna hai:** Auth chalne ke baad.

TASK
Build all DONOR screens in the mobile app and connect them to the backend.

SCREENS
1. DonorHomeScreen: greeting with name, a big availability switch card (Available / Not available), eligibility card with a progress bar and "next eligible date" if the 90-day gap is not over, small stat tiles (Requests received, Donations done), and the 3 latest incoming requests.
2. IncomingRequestsScreen: pull-to-refresh list of RequestCards (blood group badge in a red circle, urgency badge, city, distance, time ago, short message). Empty state: "No requests near you right now."
3. RequestDetailScreen: full details, a map preview of the recipient's approximate location, Accept and Reject buttons with confirm dialogs. After Accept show the recipient's name and phone with "Call" (Linking tel:) and "WhatsApp" buttons. Handle 409 ("Another donor already accepted") with a friendly message.
4. DonationHistoryScreen: completed donations with date and a thank-you note.
5. ProfileScreen (shared): edit profile, blood group, city, update location button, logout.

RULES
Use the Design System everywhere. Add loading, error and empty states for every list. Create api/requestApi.js and api/userApi.js for all calls.

ACCEPTANCE
Using seeded data, a donor can toggle availability, open a request, accept it, and call the recipient.

Check karo

- Availability toggle backend me save hota hai (app restart ke baad bhi).
- Git commit: 'step 12 donor screens'.

PROMPT 13 | Mobile: Recipient Screens and Nearby Map

Area: Mobile **Kab bhejna hai:** Donor screens ke baad.

TASK
Build all RECIPIENT screens.

SCREENS
1. RecipientHomeScreen: greeting, a large "Request Blood" primary card, EMERGENCY SOS button (wired in a later prompt, for now navigate to CreateRequest with urgency URGENT), latest request status card, and a short "How it works" strip.
2. CreateRequestScreen: patient name, BloodGroupPicker, units (stepper 1-5), urgency toggle (Normal / Urgent), city, location

block (auto-detect GPS with expo-location, show address text and a small map, allow moving the pin), required-by date/time, message. Submit calls POST /api/recipient/requests then navigates to the request details.
3. NearbyDonorsScreen: react-native-maps with donor markers + user marker, a radius chip row (5, 10, 25 km), blood group filter, and a bottom list of DonorCards (first name, blood group badge, distance such as "3.2 km away", last donated). Tapping a card highlights its marker. No phone numbers here.
4. MyRequestsScreen: list with StatusBadge and filters (All, Active, Completed).
5. RequestStatusScreen: timeline (Request sent -> Donors notified -> Donor accepted -> Completed), number of donors notified, Cancel button while PENDING, and when ACCEPTED show donor name + phone with Call button + "Mark as Completed" button.

NOTES
- Add the Google Maps API key placeholder in app.json under android.config.googleMaps.apiKey and explain how to get a key.
- Ask for location permission with a clear reason text before the system dialog.
- Handle permission denied by letting the user type a city and tap the map.

ACCEPTANCE
Recipient can create a request, see nearby donors on the map, watch the status change when a donor accepts, call the donor and mark completed.

Check karo

- Map par seeded donors (Bhopal) ke pins dikhne chahiye.
- Git commit: 'step 13 recipient screens'.

PROMPT 14 | Mobile: Location Updates, Push Notifications, Notification Screen

Area: Mobile **Kab bhejna hai:** Recipient screens ke baad. Push test karne ke liye development build chahiye (niche Common Issues dekho).

TASK
Add location syncing and push notifications to the mobile app.

LOCATION
- useLocation hook: request foreground permission, get current position, handle denied/blocked states.
- For DONORS: update the server location (PUT /api/users/location) after login and whenever the app comes to the foreground, but at most once every 30 minutes.

PUSH NOTIFICATIONS
- useNotifications hook: ask permission (Android 13+ POST_NOTIFICATIONS), get the Expo push token, send it to PUT /api/users/push-token after login, clear it on logout.
- Create two Android notification channels: "default" and "urgent" (importance MAX, vibration, red light color #DC2626).
- Foreground: show a banner. Tap on a notification: navigate to RequestDetailScreen (donor) or RequestStatusScreen (recipient) using data.requestId, even if the app was closed.

NOTIFICATION SCREEN
NotificationScreen listing in-app notifications (GET /api/notifications) with unread dot, mark one as read on tap, "Mark all read" button, and a bell icon with unread count badge in the tab headers.

ACCEPTANCE
Explain clearly how to test push on a real phone and give the exact commands. Show how to send a test push from the Expo push tool.

Check karo

- Request banate hi donor ke phone par notification aaye.
- Notification tap karne par sahi screen khule.
- Git commit: 'step 14 location push'.

PROMPT 15 | Emergency SOS Feature (Backend + Web + Mobile)

Area: Full-stack **Kab bhejna hai:** Screenshot ka red EMERGENCY SOS button yahin real kaam karega.

TASK
Make the EMERGENCY SOS button fully functional.

BACKEND
POST /api/recipient/sos with {patientName, bloodGroup, city, latitude, longitude, message}. It creates a BloodRequest with urgency URGENT, radius 25 km, requiredBy = now + 6 hours, notifies all eligible compatible donors with a high-priority push on the "urgent" channel, and returns {requestId, donorsNotified}. If no donor is found in 25 km, automatically retry once with 50 km. Limit: one active SOS per recipient at a time.

MOBILE
- SOS button on RecipientHome (red gradient, glow, alert icon).
- Tap opens a bottom-sheet confirmation: blood group picker (preselect if known), patient name, auto-detected location, a note line, and a big "Send SOS" button.
- After sending show an SOS Status screen: pulsing red circle animation, "X donors notified", live status (poll every 10 seconds), and as soon as a donor accepts show his name and phone with a Call button.
- Always show two extra quick-call buttons: "Call 112" and "Call 108" using Linking.
- Show the disclaimer: "LifeDrop connects donors and recipients. In a medical emergency, contact a hospital or call 112."

WEB
Make the navbar and hero SOS buttons open the same flow (modal) for logged-in recipients and redirect guests to Register.

ACCEPTANCE
With seeded donors, one SOS tap notifies matching donors, a donor accepts, and the recipient screen updates to show the donor contact.

Check karo

- SOS ke baad donorsNotified count sahi aa raha ho.
- Git commit: 'step 15 sos'.

PROMPT 16 | Full QA, Bug Fixing and UI Polish

Area: Full-stack **Kab bhejna hai:** Saare features banne ke baad, release se pehle.

TASK
Act as a QA engineer and a UI reviewer for the whole project. Do NOT add new features.

TEST SCENARIOS (run and report pass/fail for each)
1. Register donor, register recipient, login, logout, token expiry.
2. Donor without blood group is rejected. Donor under 18 is rejected.
3. Recipient creates NORMAL request, matching donors are notified.
4. Two donors accept at the same time: only one wins.
5. Phone number is hidden before accept and visible after accept (both sides).
6. Donor within 90 days of last donation never receives requests.
7. Request expires after 24 hours. Cancelled request cannot be accepted.
8. Rate limit (4th request in 24 hours returns 429).
9. SOS flow end to end.
10. Location permission denied path on mobile.
11. Offline / backend down: app shows a friendly error, not a crash.

UI REVIEW
Check every screen against the DESIGN SYSTEM: colors, font weights, radius, spacing, button styles, status badges, empty states and loading states. List mismatches and fix them. Make sure no text mentions hospitals, blood banks, certifications or fake numbers.

CLEANUP
Remove unused files and console logs, add missing validations, write a root README.md with: project overview, screenshots placeholders, tech stack, setup steps for backend / web / mobile, demo logins, and API overview.

OUTPUT
A table of issues found, what you fixed, and anything left open.

Check karo

- Issues table padho aur khud ek baar app chala kar verify karo.
- Git commit: 'step 16 qa and polish'.

PROMPT 17 | APK Build and Deployment

Area: Release **Kab bhejna hai:** Sab kuch test ho jaane ke baad.

TASK
Prepare the project for release.

BACKEND DEPLOYMENT
- Add a Dockerfile (multi-stage, Java 17) for the backend.
- Create application-prod.properties reading everything from environment variables.
- Give step-by-step instructions to deploy the backend on Render (or Railway) and the database on Neon (or Supabase) free tier. List every environment variable to set (DB_URL, DB_USER, DB_PASS, JWT_SECRET, JWT_EXPIRATION, SPRING_PROFILES_ACTIVE=prod).
- Restrict CORS to the real web domain in production.

MOBILE APK
- app.json: name "LifeDrop", slug "lifedrop", version 1.0.0, android.package "com.lifedrop.app", app icon and adaptive icon (red drop on white), splash screen (pink background with logo), Android permissions ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION, POST_NOTIFICATIONS.
- eas.json with profiles: development (development build), preview (APK for testing), production (AAB for Play Store).
- Set EXPO_PUBLIC_API_URL to the deployed HTTPS backend URL for preview and production.
- Give the exact commands: eas login, eas build:configure, eas build -p android --profile preview. Explain how to install the APK on a phone.

PLAY STORE READINESS
- Create a Privacy Policy page (web route /privacy) covering: what data is collected (name, phone, blood group, location), why, who can see it, deletion on request, and contact email placeholder.
- Add an in-app "Delete my account" option (backend endpoint DELETE /api/users/me that anonymizes personal data).
- Create a checklist for the Play Console (data safety form, content rating, screenshots, short and full description).

ACCEPTANCE
The backend is reachable over HTTPS, the APK installs on a real phone, login works, and push notification arrives.

Check karo

- APK dusre phone par install karke full flow test karo.
- Git tag: v1.0.0.

9. Common Issues aur Solutions

Problem	Solution
Mobile app backend se connect nahi ho rahi	Phone aur PC **same Wi-Fi** par rakho. EXPO_PUBLIC_API_URL me **PC ka LAN IP** likho (http://192.168.x.x:8080). Android emulator me http://10.0.2.2:8080. Windows Firewall me port 8080 allow karo.
Release APK me API call fail	Android release build me plain **http** block ho sakta hai. Backend ko **https** par deploy karo (Render / Railway) aur wahi URL use karo.
Push notification nahi aa rahi	Newer Expo versions me Android push **Expo Go me kaam nahi karta**. Development build ya APK use karo (eas build --profile development / preview). Real phone chahiye, emulator nahi.
Map APK me blank dikh raha hai	react-native-maps ko Android par **Google Maps API key** chahiye. app.json me android.config.googleMaps.apiKey daalo.
API 403 / 401 de rahi hai	Authorization header **Bearer token** ke saath ja raha hai ya nahi dekho. Role galat hai to bhi 403 aata hai (donor ne recipient route kholna).
Web se API call par CORS error	Backend CorsConfig me web ka URL (localhost:5173) allow karo.
Nearby donors list khali	Donors ka latitude/longitude, available = true aur 90-din gap check karo. Dev profile ka seed data use karo (Bhopal ke paas).
Antigravity purana code tod raha hai	Prompt ke shuru me likho: "Only modify files needed for this step." Har step ke baad git commit rakho.
Database connection error	PostgreSQL chal raha hai, database **lifedrop** bana hai, DB_USER / DB_PASS sahi hain, ye check karo.

10. Final Checklist

- Backend: Swagger me saari APIs chal rahi hain, tests pass.
- Phone number accept se pehle kahin bhi nahi dikhta (API response me bhi nahi).
- Donor 18+ hi register ho sakta hai, 90-din gap kaam kar raha hai.
- Do donors ek saath accept karein to sirf ek jeetta hai.
- Nearby search sahi distance deta hai aur compatible groups hi aate hain.
- SOS ek tap me donors ko notify karta hai.
- Theme har screen par same: pink background, crimson buttons, Plus Jakarta Sans.
- UI me kahin hospital / blood bank / certified jaisa text nahi.
- Medical disclaimer app me dikhta hai.
- Privacy Policy page aur Delete account option maujood hai.
- Secrets (JWT_SECRET, DB password, API keys) git me commit nahi hue.
- APK dusre phone par install karke login, request, accept aur push test kiya.

Agla kadam

- Prompt 0 se shuru karo. Jab Prompt 8 tak backend ready ho jaye, mujhe batao: main aapke liye mobile ya web ke specific bugs / screens ke liye alag prompts bana dunga.